Connect Four vs AI - Chat with Claude

Claude Sonnet 5.5 (Anthropic) | Scenario 15: terminal Connect Four, Python standard library only

Compiled prompts and replies from the lab work. The original working chat is available separately.

I'm working on a terminal Connect Four project in Python (standard library only). Files: main.py, game.py, board.py, ai.py. Here is the code: [the four original files pasted]. Before suggesting any changes, explain how a move travels from the command-line input in game.py to board.drop() and then to board.winner(). Don't change anything yet.

Claude

Here is the path of one human move in the original code:

- **main.py** calls `Game().run()`, which starts the loop in game.py.
- **game.py, run():** each loop prints the board, then, if it is X's turn, reads text with `input()`. 'q' returns. Otherwise `int(raw) - 1` converts the 1-7 the player sees into a 0-6 column index; non-numbers hit the ValueError branch and the loop repeats.
- **board.drop(col, token):** rejects columns outside 0-6, then scans rows from the bottom (row 5) to the top looking for the first '.', writes the token there and returns that row. It returns None if the column is full.
- **Back in game.py:** a None result prints 'Column unavailable.' and, for the human, repeats the turn. Otherwise it calls `board.winner(turn)`, then `board.full()` for a draw, then switches the turn.
- **board.winner(token):** visits every cell holding the token and, for each direction in `directions`, checks four consecutive cells (`range(4)`) inside the board all equal the token.

The computer's move takes the same route after `ai.choose_column()` returns a column; in the original it is just a random legal column. One thing already stands out: `directions = [(0, 1), (1, 0)]` contains only horizontal and vertical. No changes made.

In board.py, winner() uses directions = [(0, 1), (1, 0)]. Explain what bug this causes, and give me a small script that sets up a diagonal winning position for both '/' and '\' and shows winner() returns False.

Claude

Each tuple is (row step, column step). (0, 1) walks along a row and (1, 0) walks down a column, so winner() can only ever see horizontal and vertical fours. A diagonal needs both steps to change at once, (1, 1) and (1, -1), which are missing. The result: a player who completes a diagonal four is never declared the winner and the game just continues.

Script (demo_bug.py), which replays a fixed move list so the bug appears every time:

```
from board import Board

def play(moves, name):
    b = Board()
    for i, col in enumerate(moves):
        b.drop(col, "X" if i % 2 == 0 else "O")
    print(f"=== {name} ===")
    b.print()
    print("winner('X') ->", b.winner("X"))

play([0, 1, 1, 2, 6, 2, 2, 3, 6, 3, 5, 3, 3], "Diagonal /")
play([6, 5, 5, 4, 0, 4, 4, 3, 0, 3, 1, 3, 3], "Diagonal backslash")
```

Output on the **original** code:

```
=== Diagonal / ===

  1 2 3 4 5 6 7
  . . . . . . .
  . . . . . . .
  . . . X . . .
  . . X O . . .
  . X O O . . X
  X O O O . X X
winner('X') -> False
=== Diagonal backslash ===

  1 2 3 4 5 6 7
  . . . . . . .
  . . . . . . .
  . . . X . . .
  . . . O X . .
  X . . O O X .
  X X . O O O X
winner('X') -> False
```

Fix winner() so it detects all four directions (horizontal, vertical, both diagonals) without breaking existing wins, and so that a run of only 3 does not count. Keep changes minimal and explain why the new direction tuples work.

Claude

The fix is two extra direction tuples:

```
directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
#             horiz   vert   diag \   diag /
```

- **(1, 1)** moves down one row and right one column, so it walks a backslash diagonal.
- **(1, -1)** moves down one row and left one column, so it walks a slash diagonal. Because every cell is used as a starting point, scanning 'downward' covers both diagonals from their top ends.
- The check `all(... for i in range(4))` requires four in-bounds cells equal to the token, so a run of three can never count and nothing can run off the edge of the board.
- Horizontal and vertical behave exactly as before because their tuples are unchanged.

Result after the fix, same demo script:

```
=== Diagonal / ===
winner('X') -> True
=== Diagonal backslash ===
winner('X') -> True
```

Improve game.py and board.py so that draws, full columns, invalid input (letters, 0, 8, negative numbers), EOF/Ctrl+C and quitting are handled consistently. No invalid move may change the board, and a winning move must end the game before another turn is requested. Keep the existing structure.

Claude

Changes, keeping the same Game/Board structure:

- **board.drop()** now also rejects non-integers and returns None for any invalid or full-column move, without ever touching the grid. Helpers `legal_columns()` and a simpler `full()` were added.
- **Input validation in game.py:** letters give 'Invalid input...'; 0, 8 and negatives give 'Invalid column...'; 'q' or 'quit' ends the game with a clear message; EOF and Ctrl+C end the game cleanly instead of crashing.

- **Full column:** a rejected drop prints 'Column N is full' and asks again on the same turn.
- **Terminal states:** after every successful move the win check runs first and returns immediately, so no further input is requested after a win; then the draw check; only then does the turn switch. Each ending prints a clear message ('X wins!', 'Draw: the board is full.', 'You quit. Game over.').

Final board.py:

```
ROWS, COLS = 6, 7


class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        """Drop a token. Returns the row used, or None if the move is invalid
        (column out of range or full). An invalid move never changes the grid."""
        if not isinstance(col, int) or not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
        return None

    def legal_columns(self):
        return [c for c in range(COLS) if self.grid[0][c] == "."]

    def full(self):
        return not self.legal_columns()

    def copy(self):
        clone = Board()
        clone.grid = [row[:] for row in self.grid]
        return clone

    def winner(self, token):
        # horizontal, vertical, diagonal "\", diagonal "/"
        directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue
                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS
                        and 0 <= c + i * dc < COLS
                        and self.grid[r + i * dr][c + i * dc] == token
                        for i in range(4)
                    ):
                        return True
        return False

    def print(self):
        print("\n  " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print("  " + " ".join(row))
```

Final game.py (includes the Task 4 feedback line, explained in Prompt 6):

```
from board import Board, COLS, ROWS
from ai import AI


class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X. Enter 1-7 to drop a disc, q to quit.")
        while True:
            self.board.print()
```

```
if self.turn == "X":
    try:
        raw = input("Column (1-7), or q: ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print("\nInput closed. Game ended.")
        return
    if raw in ("q", "quit"):
        print("You quit. Game over.")
        return
    try:
        col = int(raw) - 1
    except ValueError:
        print("Invalid input: enter a number from 1 to 7, or q.")
        continue
    if not 0 <= col < COLS:
        print("Invalid column: choose a number from 1 to 7.")
        continue
else:
    col = self.ai.choose_column(self.board, me="O", opponent="X")
    if col is None:
        print("AI has no legal move.")
        return

row = self.board.drop(col, self.turn)
if row is None:
    print(f"Column {col + 1} is full. Pick another column.")
    continue

# Task 4: feedback once per actual placement
print(f"{self.turn} placed a disc in column {col + 1} (row {ROWS - row} from the bottom).")

# A winning move ends the game before another turn is requested
if self.board.winner(self.turn):
    self.board.print()
    print(f"{self.turn} wins!")
    return
if self.board.full():
    self.board.print()
    print("Draw: the board is full.")
    return

self.turn = "O" if self.turn == "X" else "X"
```

Improve ai.py: take an immediate winning move if one exists, otherwise block an immediate player win, otherwise avoid moves that let the opponent win next turn, and prefer centre columns. It must never choose a full column and must return None if the board is full. Add helper methods to Board (legal_columns, copy) if needed, and explain each one.

Claude

Decision order in `choose_column`:

- Get `board.legal_columns()`; if empty, return None (board full).
- **Win:** for each legal column, simulate dropping my token on a copy; if it wins, play it.
- **Block:** do the same with the opponent's token; if they would win there, play that column.
- **Safety:** avoid moves after which the opponent has an immediate winning reply, unless every move is like that.
- **Centre preference:** among the remaining columns choose those closest to column 4 (index 3), breaking ties randomly.

The two Board helpers:

- `legal_columns()` returns the columns whose top cell is empty. Everything the AI picks comes from this list, so it can never choose a full or out-of-range column.

- `copy()` returns an independent copy of the grid, so the AI can try moves for analysis without changing the real board.

Final ai.py:

```
import random


class AI:
    def _wins(self, board, col, token):
        trial = board.copy()
        return trial.drop(col, token) is not None and trial.winner(token)

    def choose_column(self, board, me="O", opponent="X"):
        legal = board.legal_columns()
        if not legal:
            return None

        # 1. Take an immediate win
        for col in legal:
            if self._wins(board, col, me):
                return col

        # 2. Block an immediate player win
        for col in legal:
            if self._wins(board, col, opponent):
                return col

        # 3. Avoid moves that hand the opponent a win on top of ours
        safe = []
        for col in legal:
            trial = board.copy()
            trial.drop(col, me)
            if not any(self._wins(trial, c, opponent) for c in trial.legal_columns()):
                safe.append(col)
        pool = safe or legal

        # 4. Prefer the centre columns
        best = min(abs(c - 3) for c in pool)
        return random.choice([c for c in pool if abs(c - 3) == best])
```

Add concise feedback after each successful disc placement in game.py, for example 'X placed a disc in column 4'. It must print once per actual move, not inside winner() or the AI's simulations. Explain why that is the right place.

Claude

The feedback line goes in `Game.run()`, immediately after `board.drop()` returns a row (so the move really happened) and before the win/draw checks:

```
row = self.board.drop(col, self.turn)
if row is None:
    print(f"Column {col + 1} is full. Pick another column.")
    continue

# Task 4: feedback once per actual placement
print(f"{self.turn} placed a disc in column {col + 1} (row {ROWS - row} from the bottom).")
```

- Rejected moves hit the `continue` above it, so they print an error, not a placement message.
- `winner()` and the AI only look at cells or work on a `copy()` of the board; neither prints, so analysis can never produce a message. The only place a real move happens is this loop, so the line prints exactly once per move.

Write a unittest file covering: horizontal, vertical, both diagonal wins, 3-in-a-row not winning, a draw position, full-column rejection, invalid columns not changing the board, AI immediate win, AI immediate block, AI never choosing a full column, quitting via a mocked input, feedback printed once per move, and a winning move stopping the game before another turn is requested.

Claude

test_connect_four.py (standard-library unittest and unittest.mock only):

```
import unittest
from unittest.mock import patch
from board import Board, ROWS, COLS
from ai import AI
from game import Game


def place(board, moves):
    for col, tok in moves:
        board.drop(col, tok)


class TestWins(unittest.TestCase):
    def test_horizontal(self):
        b = Board(); place(b, [(c, "X") for c in range(4)])
        self.assertTrue(b.winner("X"))

    def test_vertical(self):
        b = Board(); place(b, [(2, "X")] * 4)
        self.assertTrue(b.winner("X"))

    def test_diagonal_up(self):
        b = Board()
        place(b, [(0,"X"),(1,"O"),(1,"X"),(2,"O"),(6,"X"),(2,"O"),(2,"X"),
                  (3,"O"),(6,"X"),(3,"O"),(5,"X"),(3,"O"),(3,"X")])
        self.assertTrue(b.winner("X"))

    def test_diagonal_down(self):
        b = Board()
        place(b, [(6,"X"),(5,"O"),(5,"X"),(4,"O"),(0,"X"),(4,"O"),(4,"X"),
                  (3,"O"),(0,"X"),(3,"O"),(1,"X"),(3,"O"),(3,"X")])
        self.assertTrue(b.winner("X"))

    def test_three_is_not_a_win(self):
        b = Board(); place(b, [(c, "X") for c in range(3)])
        self.assertFalse(b.winner("X"))


class TestBoardEdges(unittest.TestCase):
    def test_draw(self):
        b = Board()
        a, c = "XXOOXXO", "OOXXOOX"
        b.grid = [list(a if r % 2 == 0 else c) for r in range(ROWS)]
        self.assertTrue(b.full())
        self.assertFalse(b.winner("X"))
        self.assertFalse(b.winner("O"))

    def test_full_column_rejected(self):
        b = Board()
        for i in range(ROWS):
            self.assertIsNotNone(b.drop(0, "X" if i % 2 else "O"))
        before = [r[:] for r in b.grid]
        self.assertIsNone(b.drop(0, "X"))
        self.assertEqual(b.grid, before)

    def test_invalid_columns_do_not_change_board(self):
        b = Board(); before = [r[:] for r in b.grid]
        for bad in (-1, 7, 99):
            self.assertIsNone(b.drop(bad, "X"))
        self.assertEqual(b.grid, before)


class TestAI(unittest.TestCase):
```

```
    def test_takes_win(self):
        b = Board(); place(b, [(0,"O"),(1,"O"),(2,"O"),(6,"X"),(6,"X")])
        self.assertEqual(AI().choose_column(b), 3)

    def test_blocks_win(self):
        b = Board(); place(b, [(0,"X"),(1,"X"),(2,"X"),(5,"O")])
        self.assertEqual(AI().choose_column(b), 3)

    def test_never_picks_full_column(self):
        b = Board()
        for i in range(ROWS):
            b.drop(3, "X" if i % 2 else "O")
        for _ in range(200):
            self.assertNotEqual(AI().choose_column(b), 3)

    def test_no_legal_moves(self):
        b = Board()
        b.grid = [list("XXOOXXO") for _ in range(ROWS)]
        self.assertIsNone(AI().choose_column(b))


class TestGame(unittest.TestCase):
    @patch("builtins.input", side_effect=["abc", "0", "9", "q"])
    def test_invalid_input_then_quit(self, _):
        g = Game(); g.run()
        self.assertTrue(all(c == "." for row in g.board.grid for c in row))

    @patch("builtins.input", side_effect=["q"])
    def test_quit(self, _):
        self.assertIsNone(Game().run())

    @patch("builtins.print")
    @patch("builtins.input", side_effect=["1"])
    def test_feedback_once_per_move(self, _, mock_print):
        g = Game()
        g.ai.choose_column = lambda *a, **k: 1
        with self.assertRaises(StopIteration):
            g.run()
        placed = [c for c in mock_print.call_args_list if "placed a disc" in str(c)]
        self.assertEqual(len(placed), 2)  # one for X, one for O

    @patch("builtins.input", side_effect=["1", "2", "3", "4", "5"])
    def test_win_stops_game_before_next_turn(self, mock_in):
        g = Game()
        cols = iter([0, 0, 0])
        g.ai.choose_column = lambda *a, **k: next(cols)
        # X plays 1,2,3,4 on the bottom row -> wins on 4th input
        g.run()
        self.assertEqual(mock_in.call_count, 4)


if __name__ == "__main__":
    unittest.main()
```

Run with `python3 -m unittest -v`:

```
test_blocks_win (test_connect_four.TestAI.test_blocks_win) ... ok
test_never_picks_full_column (test_connect_four.TestAI.test_never_picks_full_column) ... ok
test_no_legal_moves (test_connect_four.TestAI.test_no_legal_moves) ... ok
test_takes_win (test_connect_four.TestAI.test_takes_win) ... ok
test_draw (test_connect_four.TestBoardEdges.test_draw) ... ok
test_full_column_rejected (test_connect_four.TestBoardEdges.test_full_column_rejected) ... ok
test_invalid_columns_do_not_change_board (test_connect_four.TestBoardEdges.test_invalid_columns_do_not_change_board)
test_feedback_once_per_move (test_connect_four.TestGame.test_feedback_once_per_move) ... ok
test_invalid_input_then_quit (test_connect_four.TestGame.test_invalid_input_then_quit) ... ok
test_quit (test_connect_four.TestGame.test_quit) ... ok
test_win_stops_game_before_next_turn (test_connect_four.TestGame.test_win_stops_game_before_next_turn) ... ok
test_diagonal_down (test_connect_four.TestWins.test_diagonal_down) ... ok
test_diagonal_up (test_connect_four.TestWins.test_diagonal_up) ... ok
test_horizontal (test_connect_four.TestWins.test_horizontal) ... ok
test_three_is_not_a_win (test_connect_four.TestWins.test_three_is_not_a_win) ... ok
test_vertical (test_connect_four.TestWins.test_vertical) ... ok

----------------------------------------------------------------------
Ran 16 tests in 0.123s

OK
```

Here is my final code: [board.py, ai.py, game.py pasted]. Review it for edge cases I missed, and confirm I added no external dependencies and no persistent storage.

Claude

Dependencies and storage: only `random`, `unittest` and `unittest.mock` from the standard library are used. All state lives in memory (`Board.grid`); nothing is written to files or databases.

Edge cases checked:

- Win on the very last empty cell: the win check runs before the draw check, so it is reported as a win, not a draw.
- Input like ' 4 ' or 'Q' works (stripped and lower-cased); '4.0' and 'abc' are rejected as invalid input; a huge number like 10**100 is rejected as out of range.
- Ctrl+C or EOF at the prompt exits cleanly.
- AI on a full board returns None; in normal play this cannot happen because the draw check ends the game first.
- AI never mutates the real board (it only uses copies).

Minor observations, not bugs:

- `isinstance(col, int)` accepts True/False because bool is an int in Python; the game loop only passes ints from `int()`, so this cannot be reached through play.
- winner() rescans the whole 6x7 board each move; this is trivially fast at this size, and a check from the last move would be a possible optimisation.
- The AI looks one move ahead only. It handles all immediate wins and blocks but not two-move traps, which is outside the lab's scope.